

CSE234 Week 2: Arrays, STL Array, Command Line Arguments

Containers are an important data structure in computer structure. Containers are abstractions that store multiple data elements. The most basic container is an array. Arrays store a collection of data items of the same type. The data is stored contiguously in memory. There are two primitive types of array in C++, static and dynamic. Static arrays are declared like `int arr[10];`, and the size of a static array needs to be known at compile time. Dynamic arrays use pointers, and the size can be determined at runtime. After reviewing static and dynamic arrays, we'll begin discussing the C++ Standard Template Library (STL) and its array class. Afterwards, we'll discuss command line arguments.

Static and Dynamic Arrays

Static arrays are used when the size of the array is known at compile time (i.e., when you are writing the code). Below is an example of a static array:

```
// declaring the array
int values[10];

// inserting values into it
for (int i = 0; i < 10; ++i) {
    values[i] = i;
}
```

Static arrays are declared by putting square brackets with the size of the

array in them at the end of the name. The square brackets can be passed an integer to access that element of the array. Arrays are 0-indexed, so the first element in the array is index 0, the second element is index 1, etc. The last element of the array is the one less than its size. For loops work well for iterating through the individual indices of the array. Accessing values that are out of bounds is undefined behavior in C++. It will not throw an exception, but it will read garbage values. If the size of the array is not known at compile time, a dynamic array can be used. A dynamic array uses a pointer and allocates memory using the `new` operator. Below is an example:

```
// read in the size
int num_scores;
std::cout << "Enter the number of scores: ";
std::cin >> num_scores;

// declare a pointer and pass the
// size to the new operator
int *scores = new int[num_scores];

// afterwards, use it like a normal array
for (int i = 0; i < num_scores; ++i) {
    std::cout << "Enter a score: ";
    std::cin >> scores[i];
}

// dynamic arrays need to be freed with delete[]
delete[] scores;
```

Dynamic arrays are in the same way as a static array after they have been declared. However, when you are done with the array, it needs to be deleted. `delete[]` with the array brackets is used to free dynamic arrays. Failure to free memory can lead to memory leaks. Memory leaks will be discussed later in the course.

STL Array

The Standard Template Library (STL) provides implementations of most

common data structures. These include arrays, vectors, lists, sets, maps, stacks, and queues. All of these have dynamically changing size, except for array, whose size must be known and specified at compile-time. Below is a brief overview of each:

- **Array** - A fixed-size collection of elements that preserves order and allows random access (index)
- **Vector** - An automatically resizing collection that preserves order and allows random access (index)
- **List** - A collection that does not support random access but does preserve order
- **Set** - A collection that does not support random access. Elements of a set are unique
- **Multiset** - A set where items are not unique, but there is a count of how many times an element occurs
- **Map** - A collection of (key, value) pairs with unique keys
- **Multimap** - A collection of (key, value) pairs where the keys are not unique
- **Stack** - A container where items can only be accessed in last-in, first-out (LIFO) order (like a stack of dishes, the last dish put on the top of the stack is the first one to be removed)
- **Queue** - A container where items can only be accessed in first-in, first-out (FIFO) order (like a line (queue) waiting to check out at a store)

This week, we'll discuss the array class. The array is a class wrapping a static array, so the size must be known at compile time like a normal static array. Vectors, which will be discussed next week, are dynamically resizing arrays.

Array

`std::array`, and all STL classes, are template classes. We'll discuss how to write templates towards the end of the course, but using them is simple. When declaring an instance of the class, the type is followed

by a set of angle brackets. For `std::array`, the data type the array stores and the size of the array are put between the angle brackets. This is called the template argument list. For example, to declare an array with 10 integers, we would write the following code:

```
std::array<int, 10> values;
```

To use `std::array` we must include the `array` header.

Accessing Elements

There are two ways to access the elements of an array. You can use the subscript operators, just like you would for a normal C array, or you can use the `at()` member function. The `at()` function will throw an exception if you try to access an out-of-bounds element. For example:

```
std::array<int, 5> values;

// this is undefined behavior
// this illegal write may not cause
// an immediate problem, but in a large
// program it can cause a crash later on
values[5] = 5;
// this will throw an std::out_of_range exception
// since 5 is not a valid index
values.at(5) = 5;
```

The `at()` member function can be used for both reading and writing to values of the array.

There are also `front()` and `back()` members which return the first and last element, respectively.

Size

There are two member functions for accessing the size of the array:

`size()` and `max_size()`. `max_size()` returns the actual number of elements the array is capable of holding, while `size()` returns the

highest index that has had data written to it. The example will illustrate:

```
std::array<int, 10> values;
for (int i = 0; i < 5; ++i) {
    values[i] = i;
}

// this will output 5
std::cout << values.size() << std::endl;
// this will output 10
std::cout << values.max_size() << std::endl;

values[9] = 9;
// this will output 10
std::cout << values.size() << std::endl;
```

Example

```
#include <iostream>
// needed to use std::array
#include <array>

int main() {
    // an array can be declared and provided an initializer
    // just like a normal array can
    std::array<int, 5> scores = {10, 20, 30, 40, 50};

    // size() member function
    std::cout << "The array contains " << scores.size() << " elements."

    // accessing elements with the subscript is faster but undefined
    // if the index passed is invalid
    std::cout << "First element (using []): " << scores[0] << "\n";

    // at() is safer but slower
    std::cout << "Third element (using .at()): " << scores.at(2) << "\n";

    // modify elements the same as a static array
    scores[4] = 100;

    // can be iterated with a traditional for loop
    // or a range-based for loop
    std::cout << "All scores: ";
```

```

    for (int score : scores) {
        std::cout << score << " ";
    }
    std::cout << "\n";

    double total = 0;
    for (int i = 0; i < scores.size(); ++i) {
        total += scores.at(i);
    }

    std::cout << "The average is " << total / scores.size() << std::endl;

    return 0;
}

```

Command Line Arguments

Command line arguments are an array of C-strings that are provided to a program when run from the command line. Even desktop applications typically have the ability to accept command line arguments when they are run. Command line args are an important mechanism for passing data to a program. Most command line programs do not have interactive user input. Instead, they read all the data they expect from the user from the command line args.

To access them, you need to add two parameters to your main function:

```

int main(int argc, char **argv) {}
// alternatively, you can do this for argv
int main(int argc, char *argv[]) {}
// or even this
int main(int argc, char argv[][] ) {}

```

The argument, `argc` is the number of command line arguments passed to the program. The argument `argv` is an array of C-strings (and C-strings are also arrays, so it is an array of arrays) that contain the actual arguments. Every program has at least one command line argument, which is the command used to run the program itself. So, to access the

actual arguments to your program, you will need to start at the index of 1. Below is an example:

```
int main(int argc, char **argv) {

    std::cout << "The name of the program is "
        << argv[0] << std::endl;

    std::cout << "The remaining arguments are: " << std::endl;
    // start for loop at i = 1 to skip program name
    for (int i = 1; i < argc; ++i) {
        std::cout << argv[i] << std::endl;
    }

    return 0;
}
```

Command line arguments are separated by spaces, so if we provided:

```
apple orange banana
```

as our command line args, and the name of the program was "fruit", then to run the program with those arguments we would do:

```
./fruit apple orange banana
```

in the terminal (many IDEs have ways to pass command line arguments for you).

The final output for this program would be:

```
The name of the program is `./fruit`
The remaining command line arguments are:
apple
orange
banana
```

Conclusion

This week, we reviewed static and dynamic arrays. We also introduced the STL Array class. Finally, we discussed command line arguments as an alternate mechanism for reading user input.

Documentation built with [MkDocs](#).